

操作系统实验二 进程同步与互斥

实验目的

1.掌握进程同步和互斥原理，理解生产者-消费者模型；

实验内容

实验内容1

依据生产者 - 消费者模型，在Windows环境下创建一个控制台进程，在该进程中创建n个线程模拟生产者和消费者，实现进程(线程)的同步与互斥，分析、熟悉生产者消费者问题仿真的原理和实现技术。学习Windows的线程控制和信号量、临界区工具的使用。

#生产者-消费者模型:

#full 信号量控制缓冲区的数据数量。

#mutex 保护对共享缓冲区的访问。

```
import threading # 导入线程模块
```

```
import time # 导入时间模块
```

```
import random # 导入随机模块
```

```
BUFFER_SIZE = 5 # 缓冲区大小
```

```
buffer = [] # 缓冲区
```

```
# 信号量
```

```
empty = threading.Semaphore(BUFFER_SIZE) # 空位信号量
```

```
full = threading.Semaphore(0) # 数据信号量
```

```
mutex = threading.Lock() # 互斥锁
```

```
def producer(producer_id):
```

```
    while True:
```

```
        time.sleep(random.uniform(0.5, 2)) # 模拟生产耗时
```

```
        item = random.randint(1, 100) # 生产的物品
```

```
        empty.acquire() # 等待空位
```

```
        with mutex: # 进入临界区
```

```
            buffer.append(item) # 放入缓冲区
```

```
            print(f"Producer {producer_id} has produced: {item}")
```

```
        full.release() # 增加数据信号量
```

```
def consumer(consumer_id):
```

```
    while True:
```

```
        time.sleep(random.uniform(0.5, 3)) # 模拟消费耗时
```

```
        full.acquire() # 等待数据
```

```
        with mutex: # 进入临界区
```

```
            item = buffer.pop(0) # 取出缓冲区的物品
```

```
            print(f"Consumer {consumer_id} has consumed: {item}")
```

```
        empty.release() # 增加空位信号量
```

```
# 创建生产者和消费者线程
```

```
producers = [threading.Thread(target=producer, args=(i,)) for i in range(2)]
```

```
consumers = [threading.Thread(target=consumer, args=(i,)) for i in range(3)]
```

```
for p in producers:
```

```
    p.start()
```

```
for c in consumers:
    c.start()

for p in producers:
    p.join()
for c in consumers:
    c.join()
```

实验内容2

有两组并发进程：读者和写者，共享一个文件F，编写读写者仿真程序

要求：

- (1)允许多个读者可同时对文件执行读操作；
- (2)只允许一个写者往文件中写信息；
- (3)任一写者在完成写操作之前不允许其他读者或写者工作；
- (4)写者执行写操作前，应需已有的写者和读者全部退出。
- (5)要求仿真程序产生3个读者进程，两个写者进程，读写者都周期性地产生读写要求，读写操作要持续一定时间。

```

#读者-写者模型:
#mutex 保护对 reader_count 的修改。
#write_lock 确保写者独占访问。

import threading
import time
import random

# 信号量和计数器
mutex = threading.Lock() # 保护 reader_count 的锁
write_lock = threading.Semaphore(1) # 写者锁。如果大于0则减1，让写者获得访问权限；如果为0，线程将被阻塞
reader_count = 0 # 当前读者数量

def reader(reader_id):
    global reader_count
    while True:
        time.sleep(random.uniform(0.5, 2)) # 模拟读取请求时间

        with mutex: # 修改 reader_count 的临界区
            reader_count += 1
            if reader_count == 1: # 第一个读者阻止写者
                write_lock.acquire()
        print(f"Reader {reader_id} is reading.")
        time.sleep(random.uniform(0.5, 1.5)) # 模拟读操作

        with mutex: # 修改 reader_count 的临界区
            reader_count -= 1
            if reader_count == 0: # 最后一个读者释放写者
                write_lock.release()

def writer(writer_id):
    while True:
        time.sleep(random.uniform(1, 3)) # 模拟写入请求时间

        write_lock.acquire() # 写者需要独占访问
        print(f"Writer {writer_id} is writing.")
        time.sleep(random.uniform(1, 2)) # 模拟写操作
        write_lock.release() # 释放写者锁

# 创建读者和写者线程
readers = [threading.Thread(target=reader, args=(i,)) for i in range(3)]
writers = [threading.Thread(target=writer, args=(i,)) for i in range(2)]

```

```
for r in readers:  
    r.start()  
for w in writers:  
    w.start()
```

```
for r in readers:  
    r.join()  
for w in writers:  
    w.join()
```